

Listado de funciones y clases usados en la práctica 4

Biblioteca Unified-Planning

Módulo `io`

- Clase `PDDLReader`: lectura de ficheros PDDL.
- Clase `PDDLWriter`: escritura de ficheros PDDL.

Módulo `shortcuts`

- Clase `ObjectType`: especifica un tipo de objeto del dominio. Argumentos: `name`, nombre del tipo de objeto, y, opcionalmente, `father`, para indicar que se trata de un subtipo de ese tipo de objetos.
- Clase `Fluent`: especifica un fluente del dominio, generalización del concepto de predicado.
 - Clase `BoolType`: implementa el tipo booleano para los fluentes.
 - Clase `IntType`: implementa el tipo entero para los fluentes.
 - Clase `Int`: construye constantes de tipo entero.
- Clase `InstantaneousAction`: especifica una acción instantánea del dominio.
 - Método `add_precondition`: añade a la acción la precondition proporcionada.
 - Método `add_effect`: añade a la acción el efecto proporcionado.
- Clase `Object`: especifica un objeto concreto de un cierto tipo.
- Clase `Problem`: especifica un problema de planificación clásica.

- Atributo `user_types`: proporciona una lista con los tipos de objeto del problema.
- Método `add_fluent`: añade al problema el fluente proporcionado.
- Método `add_actions` añade al problema cada acción de la lista proporcionada.
- Método `set_initial_value`: establece el valor inicial del fluente proporcionado.
- Método `add_goal`: añade al problema el objetivo proporcionado.
- Método `add_quality_metric`: añade al problema la métrica proporcionada.
- Atributo `kind`: proporciona el tipo del problema, en función de las características que este utilice.
- Método `clone`: devuelve una copia del problema.
- Clase `OneshotPlanner`: planificador en modo *oneshot*.
 - Método `solve`: proporciona, si existe, un plan solución al problema proporcionado.
 - Método `supports`: determina la compatibilidad del planificador con el tipo de problema proporcionado.
- Clase `MinimizeActionCosts`: implementa una métrica de minimización de costes de acciones.